Adding Type Annotations to Functions

Functions also get to participate in the static-typing fun. The parameters and return values can have type annotations, and the function as a whole has its own static type.

Annotating Function Parameters

TypeScript function parameters can have type annotations, with a similar syntax to what we've seen for variable declarations. The annotations work whether the function is a named function, an anonymous function, or a method of a class. All three of these examples have the same typing:

```
function priceOfTicket(row: number, accessType: string) : number { }

let priceOfTicket = function(row: number, accessType: string) : number { }

class Ticket {
  priceOfTicket(row: number, accessType: string) : number { }
}
```

In all three cases, the function `priceOfTicket` expects two arguments: a number first and a string second. It then returns a number.

Let's talk about the return type first. As currently written, all three of these functions would fail compilation because they claim to return a number, but at the moment they don't return anything. The TypeScript compiler will not compile a function that sets a return type but does not return a value.

If you want to claim explicitly that the function will not return a value, you can use the special type `void`, which means "no value":

```
function sendAnAlert(message: string) : void { }
```

Now you get the opposite behavior from the compiler—if you try to `return` a value from a `void` function, the compiler will complain.

If you don't explicitly specify the return type of the function, you still get